LAPORAN PRAKTIKUM BACKEND DEVELOPMENT

Pertemuan 6 — Modular Monolith dan Domain-Driven Design

Sistem Perpustakaan Berbasis Laravel

Disusun oleh:

Nama	[Nama Mahasiswa]
NIM	[NIM]
Kelas	[Kelas]
Mata Kuliah	Backend Development
Dosen Pengampu	[Nama Dosen]
Tanggal	7 November 2026
Repository GitHub	[URL Repository GitHub]

BAB I — PENDAHULUAN

1.1 Latar Belakang

Aplikasi perpustakaan melibatkan beberapa area bisnis yang berbeda: pengelolaan data buku, pengelolaan anggota, serta proses peminjaman dan pengembalian. Pada praktik sebelumnya, seluruh logika sering ditempatkan dalam satu struktur yang bercampur. Ketika jumlah fitur bertambah, pendekatan tersebut menyulitkan pemeliharaan karena batas tanggung jawab antar bagian menjadi kabur.

Praktikum Pertemuan 6 memperkenalkan **modular monolith** dan **Domain-Driven Design (DDD)**. Aplikasi tetap berjalan sebagai satu kesatuan (satu proses Laravel dan satu database), tetapi kode dipisahkan menjadi beberapa **bounded context** dengan tanggung jawab yang jelas. Setiap context memiliki model, migration, route, dan service sendiri. Komunikasi lintas context dibatasi melalui **contract** sehingga ketergantungan antar modul tetap terkendali.

Sistem yang dibangun pada laporan ini adalah **Sistem Perpustakaan** dengan tiga bounded context: **Catalog** (buku, penulis, kategori, penerbit), **Membership** (anggota, role, keanggotaan), dan **Circulation** (peminjaman, pengembalian, denda). Aplikasi juga menerapkan relasi database lanjutan, optimasi query, transaksi atomik, soft delete, domain event, dan Value Object.

1.2 Rumusan Masalah

1. Bagaimana merancang struktur modular monolith dengan pemisahan bounded context yang jelas?
2. Bagaimana mengimplementasikan relasi Eloquent lanjutan (Many-to-Many dengan pivot, self-referencing, dan HasMany) pada sistem perpustakaan?
3. Bagaimana menjaga konsistensi data saat peminjaman buku menggunakan transaksi database?
4. Bagaimana mengoptimalkan query untuk menghindari masalah N+1?
5. Bagaimana menerapkan soft delete, restore, domain event, dan Value Object dalam studi kasus nyata?

1.3 Tujuan

1. Membangun aplikasi Laravel dengan struktur modular per bounded context.
2. Mengimplementasikan relasi database lanjutan beserta bukti pada antarmuka.
3. Menerapkan transaksi atomik dan rollback pada proses peminjaman.
4. Mengukur dan mengurangi jumlah query menggunakan eager loading dan query scope.
5. Menerapkan soft delete/restore, domain event, Value Object ISBN, dan Value Object Denda.
6. Menyediakan pengujian otomatis untuk memverifikasi seluruh aturan bisnis utama.

1.4 Manfaat

- **Bagi mahasiswa:** memahami penerapan DDD dan modular monolith pada proyek Laravel nyata.
- **Bagi pengguna:** memperoleh aplikasi perpustakaan yang mencatat peminjaman, stok, denda, dan riwayat secara konsisten.
- **Bagi pengembangan selanjutnya:** struktur modul memudahkan penambahan fitur tanpa mengubah seluruh basis kode.

1.5 Batasan

- Autentikasi menggunakan Laravel Breeze yang sudah tersedia; fokus praktikum pada tiga bounded context perpustakaan.
- Database default menggunakan SQLite; skema dirancang agar tetap kompatibel dengan MySQL/PostgreSQL.
- Pembayaran denda hanya menandai status lunas (belum terintegrasi payment gateway).
- Notifikasi domain event ditulis ke file log, belum dikirim melalui email/queue.

BAB II — LANDASAN TEORI

2.1 Modular Monolith

Modular monolith adalah aplikasi yang di-deploy sebagai satu unit, tetapi kode internalnya dibagi menjadi modul-modul dengan batas tanggung jawab yang tegas. Keuntungannya: operasional sederhana seperti monolith, namun keteraturan kode mendekati arsitektur mikroservis.

Pada Laravel, modularitas diterapkan melalui namespace `Modules\*`, service provider per modul, serta pemisahan migration, route, dan view.

2.2 Domain-Driven Design dan Bounded Context

DDD menekankan pemodelan perangkat lunak berdasarkan bahasa dan aturan domain. **Bounded context** adalah batas di mana sebuah model dan istilah memiliki makna yang konsisten. Pada sistem ini:

- **Catalog** berbicara tentang buku, stok, penulis, penerbit, dan kategori.
- **Membership** berbicara tentang anggota, role, dan masa keanggotaan.
- **Circulation** berbicara tentang peminjaman, pengembalian, denda, dan status peminjaman.

Karena `Book` adalah milik Catalog dan `Member` adalah milik Membership, context Circulation tidak mengakses model tersebut secara langsung. Circulation memakai `BookServiceInterface` dan `MemberServiceInterface` yang didefinisikan di **Shared Kernel**.

2.3 Relasi Eloquent

- **One-to-Many / HasMany:** satu peminjaman memiliki banyak item (`Loan::items()`).
- **Many-to-Many:** buku memiliki banyak kategori dan kategori memiliki banyak buku, dengan pivot `book_category` yang menyimpan `added_at`.
- **Self-referencing:** kategori dapat memiliki induk dan anak (`Category::parent()` dan `Category::children()`).

2.4 N+1 Query dan Eager Loading

N+1 terjadi saat relasi diakses di dalam loop sehingga menambah satu query per baris. Eager loading (`with()`) memuat relasi dalam satu query tambahan sehingga jumlah query tidak tumbuh linear terhadap jumlah data.

2.5 Transaksi Database

Transaksi memastikan sekumpulan operasi diperlakukan sebagai satu kesatuan (ACID). Jika satu operasi gagal, seluruh perubahan dibatalkan. Laravel menyediakan `DB::transaction()` dan `lockForUpdate()` untuk mengunci baris selama transaksi.

2.6 Soft Delete

Soft delete tidak menghapus baris secara fisik, melainkan mengisi kolom `deleted_at`. Eloquent menyediakan trait `SoftDeletes`, method `withTrashed()`, `onlyTrashed()`, dan `restore()`.

2.7 Domain Event

Domain event adalah objek yang menandai terjadinya peristiwa bisnis, misalnya `BookBorrowed`. Event dipisahkan dari aksi utamanya; listener menangani efek samping seperti pencatatan log.

2.8 Value Object

Value Object adalah objek immutable yang divalidasi saat dibuat dan tidak memiliki identitas sendiri. Pada sistem ini terdapat `ISBN` (validasi checksum) dan `Denda` (perhitungan otomatis).

BAB III — ANALISIS DAN PERANCANGAN

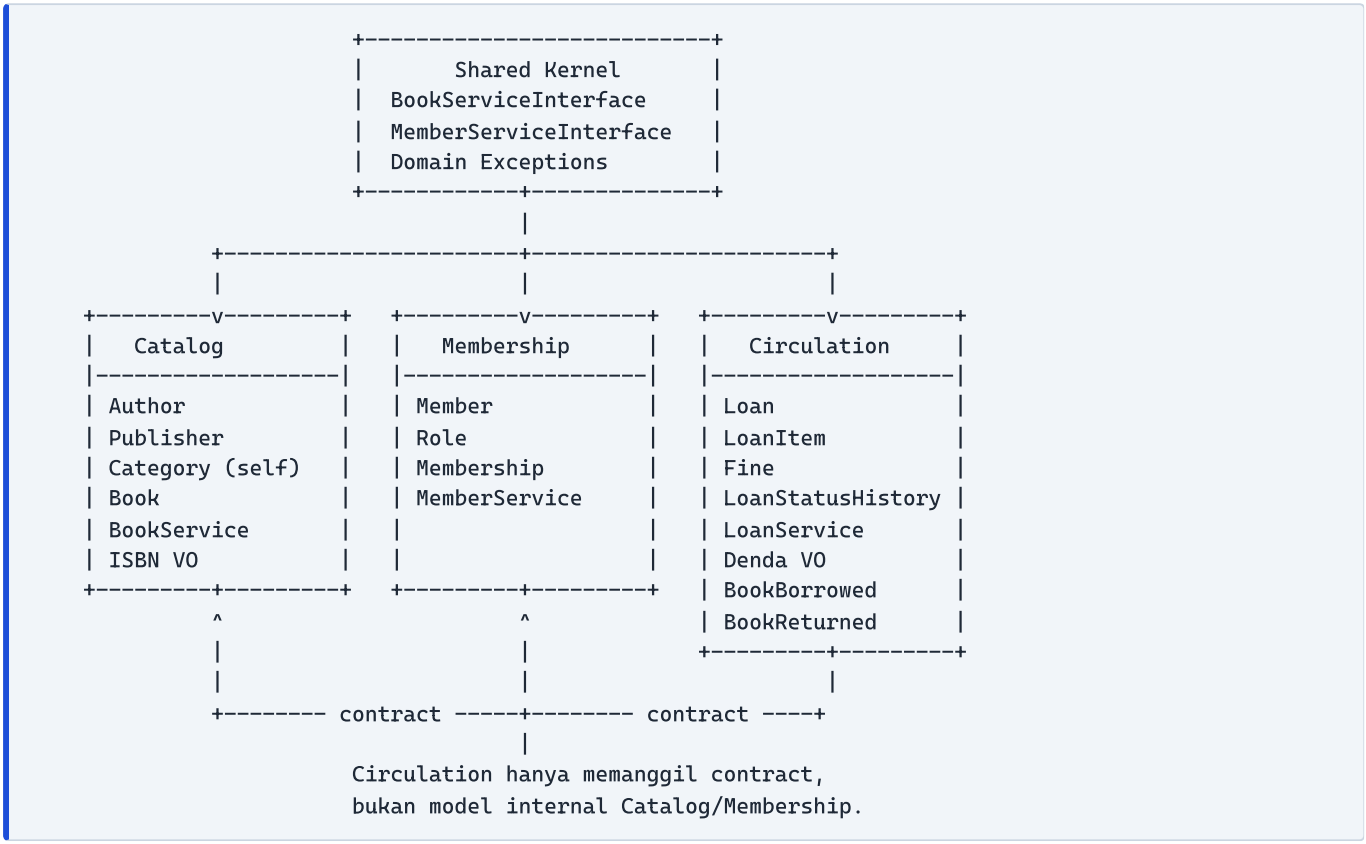
3.1 Kebutuhan Fungsional

No	Kebutuhan	Context	Status
F-01	CRUD buku, penulis, penerbit, kategori	Catalog	Selesai
F-02	Pencarian dan filter buku (kategori, penulis, penerbit, ketersediaan)	Catalog	Selesai
F-03	Validasi ISBN-10/ISBN-13	Catalog	Selesai
F-04	CRUD anggota, role, dan riwayat keanggotaan	Membership	Selesai
F-05	Pengecekan kelayakan anggota sebelum meminjam	Membership	Selesai
F-06	Pembuatan peminjaman beserta detail item	Circulation	Selesai
F-07	Pengurangan stok otomatis dan rollback bila gagal	Circulation + Catalog	Selesai
F-08	Pengembalian buku dan pengembalian stok	Circulation + Catalog	Selesai
F-09	Perhitungan denda otomatis	Circulation	Selesai
F-10	Pembayaran denda	Circulation	Selesai
F-11	Soft delete dan restore buku/anggota/peminjaman	Semua context terkait	Selesai
F-12	Log domain event peminjaman/pengembalian	Circulation	Selesai

3.2 Kebutuhan Non-Fungsional

Aspek	Target
Keamanan	Route tulis hanya untuk `role:admin`; validasi request di server
Konsistensi	Transaksi ACID + row lock pada stok dan status anggota
Performa	Eager loading pada seluruh halaman daftar; query scope untuk filter
Maintainability	Batas context dijaga contract; service provider per modul
Testability	Unit test Value Object dan feature test alur bisnis
Auditability	Snapshot data, riwayat status, dan log event

3.3 Arsitektur Bounded Context



3.4 Struktur Modul

```
app/Modules/
├── Shared/
│   ├── Contracts/BookServiceInterface.php
│   ├── Contracts/MemberServiceInterface.php
│   └── Exceptions/
├── Catalog/
│   ├── Controllers/  Models/  Requests/  Rules/
│   ├── Services/BookService.php
│   ├── ValueObjects/ISBN.php
│   ├── Migrations/  Providers/  routes/
├── Membership/
│   ├── Controllers/  Models/  Requests/
│   ├── Services/MemberService.php
│   ├── Migrations/  Providers/  routes/
└── Circulation/
    ├── Controllers/  Models/  Requests/
    ├── Events/  Listeners/
    ├── Services/LoanService.php
    ├── ValueObjects/Denda.php
    ├── Migrations/  Providers/  routes/
```

Setiap provider memanggil `loadMigrationsFrom()` dan `loadRoutesFrom()`. Provider didaftarkan pada `bootstrap/providers.php`. Binding contract dilakukan di `AppServiceProvider`:

```
$this->app->bind(BookServiceInterface::class, BookService::class);
$this->app->bind(MemberServiceInterface::class, MemberService::class);
```

3.5 Perancangan Database

Ringkasan tabel:

Tabel	Context	Keterangan
`authors`	Catalog	Penulis buku
`publishers`	Catalog	Penerbit buku
`categories`	Catalog	Kategori dengan `parent_id` (self-referencing)
`books`	Catalog	Data buku, stok, ISBN unik, soft delete
`book_category`	Catalog	Pivot Many-to-Many + `added_at`
`roles`	Membership	Role anggota
`members`	Membership	Anggota, status, kuota pinjaman, soft delete
`memberships`	Membership	Riwayat keanggotaan
`member_role`	Membership	Pivot Many-to-Many + `assigned_at`
`loans`	Circulation	Header peminjaman, snapshot nama anggota, soft delete
`loan_items`	Circulation	Detail buku yang dipinjam, snapshot judul/ISBN
`fines`	Circulation	Denda keterlambatan
`loan_status_histories`	Circulation	Audit trail perubahan status

Relasi utama:

```
authors 1 ---- * books * ---- * categories
                  |                 |
                  |                 +-- parent_id (self)
                  |
publishers 1 --- * books

members 1 ---- * loans 1 ---- * loan_items * ---- 1 books
   |              |
   |              +---- * fines
   |              +---- * loan_status_histories
   +---- * memberships
   +---- * member_role * ---- 1 roles
```

Kardinalitas dan bukti implementasi:

Relasi	Implementasi	Bukti UI
Book ↔ Category (M:N)	`belongsToMany` + `withPivot('added_at')`	Detail buku menampilkan `added_at`; form buku menampilkan checkbox kategori
Category → sub-category	`parent()` / `children()`	Kolom Induk dan Sub-kategori pada halaman kategori
Loan → LoanItem (1:N)	`hasMany`	Detail peminjaman menampilkan daftar item
Member ↔ Role (M:N)	`belongsToMany` + `withPivot('assigned_at')`	Detail anggota menampilkan checkbox role
Member → Membership (1:N)	`hasMany`	Detail anggota menampilkan riwayat keanggotaan

3.6 Alur Peminjaman

```
Admin
  |
  v
LoanController@store
  |
  v
LoanService@borrow  (DB::transaction)
  |
  +--> MemberService: getMemberForLoan, isEligibleToBorrow
  |
  +--> Loan::create (snapshot member_name, due_at = now + 7 hari)
  |
  +--> untuk setiap item:
  |      BookService: getBookForLoan, isStockAvailable
  |      BookService: reduceStock (lockForUpdate + decrement)
  |      LoanItem::create (snapshot book_title, book_isbn)
  |
  +--> MemberService: markAsBorrowing (active_loans + 1)
  |
  +--> LoanStatusHistory::create
  |
  +--> event(BookBorrowed) -> LogBookBorrowed
  |
  v
COMMIT -> redirect detail
```

Jika stok tidak cukup, exception dilempar dan seluruh perubahan di-rollback.

3.7 Matriks Akses

Halaman/Aksi	Admin	User
Daftar & detail buku/kategori/penulis/penerbit	✅	✅
Tambah/edit/hapus/restore data master	✅	❌ 403
Daftar & detail anggota/role	✅	✅
Tambah/edit/hapus anggota, role, keanggotaan	✅	❌ 403
Daftar & detail peminjaman/denda	✅	✅
Buat/kembalikan/hapus/restore peminjaman, bayar denda	✅	❌ 403

3.8 Rancangan Pengujian

Level	Fokus	File
Unit	ISBN dan Denda	`tests/Unit/IsbnTest.php`, `tests/Unit/DendaTest.php`
Feature	Transaksi peminjaman, rollback, event	`tests/Feature/Library/LoanTransactionTest.php`
Feature	Soft delete/restore dan otorisasi	`tests/Feature/Library/SoftDeleteRestoreTest.php`
Feature	Jumlah query dan scope	`tests/Feature/Library/QueryOptimizationTest.php`
Feature	Smoke test seluruh halaman	`tests/Feature/Library/PageSmokeTest.php`

BAB IV — IMPLEMENTASI DAN PENGUJIAN

4.1 Lingkungan Pengembangan

Komponen	Versi
PHP	8.5.0
Laravel	13.38.0
Composer	2.8.12
Node.js	24.18.0
Database	SQLite
Browser	Chromium/Edge

4.2 Langkah Menjalankan

```
composer install
php artisan migrate:fresh --seed
npm install
npm run build
php artisan serve
```

Akun demo:

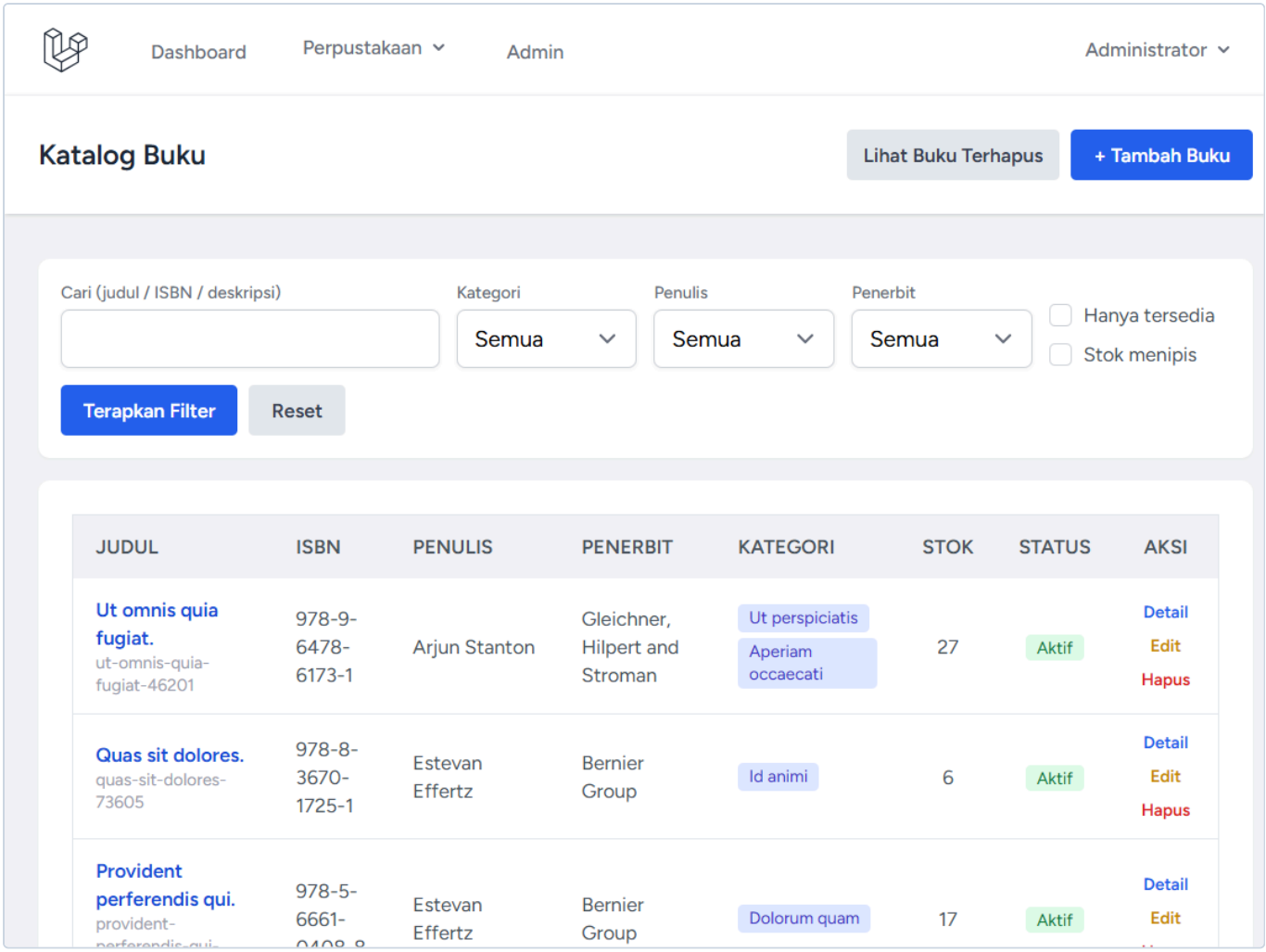
- Admin: `admin@example.com` / `password123`
- User: `user@example.com` / `password123`

Seeder menghasilkan 40 buku, 15 anggota, 11 peminjaman, 12 item, dan 2 denda.

4.3 Implementasi Catalog

4.3.1 Katalog Buku

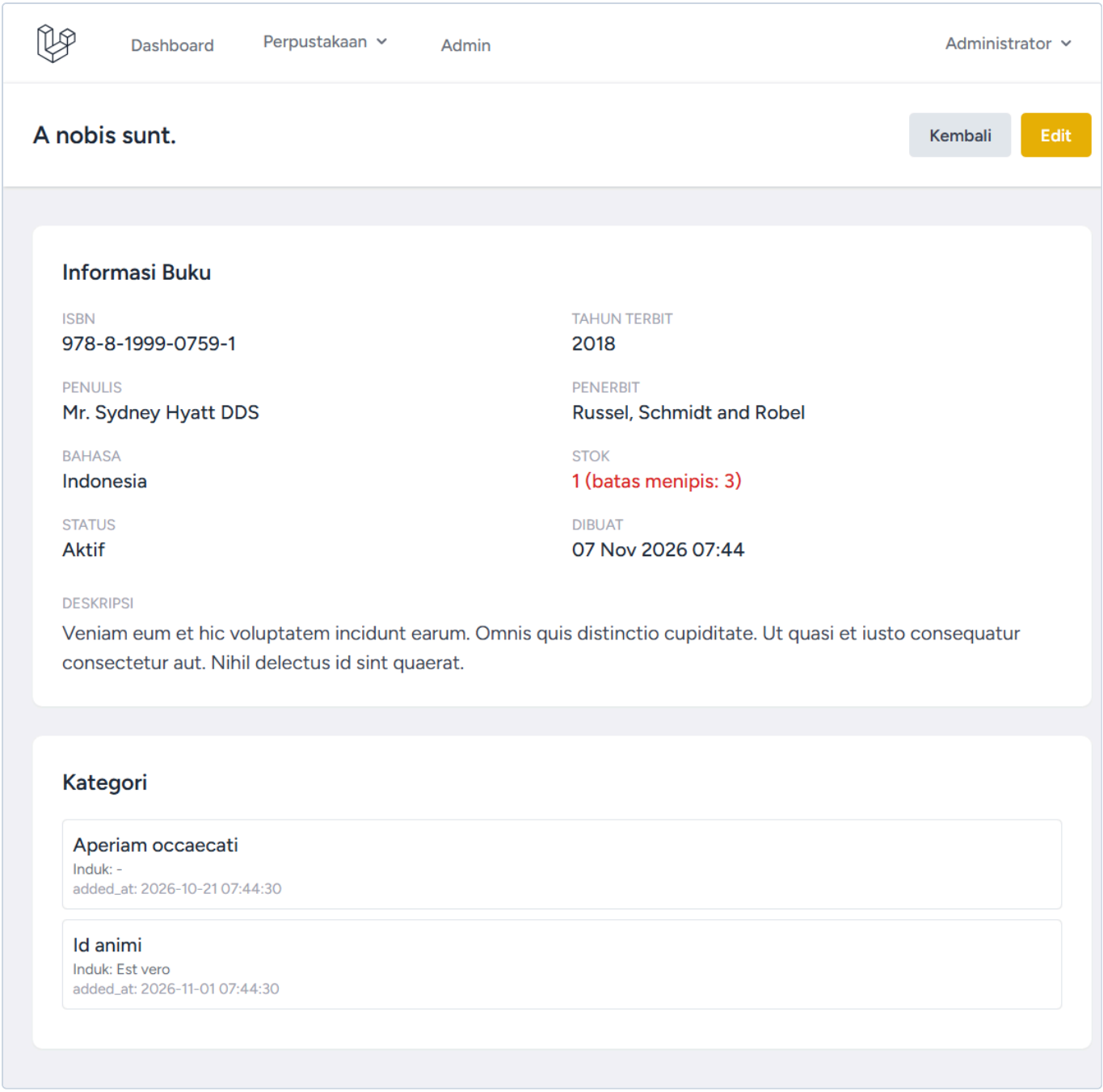
Halaman katalog menampilkan relasi penulis, penerbit, dan kategori. Pencarian/filter menggunakan query scope dan seluruh relasi di-eager load untuk menghindari N+1.



Gambar 4.1 — Daftar buku dengan filter pencarian, kategori, penulis, penerbit, dan status stok.

4.3.2 Detail Buku

Detail buku menampilkan ISBN terformat, stok (berwarna merah bila di bawah ambang), serta daftar kategori beserta `added_at` dari pivot.



Gambar 4.2 — Detail buku; kategori menampilkan relasi Many-to-Many dan pivot `added_at`.

4.3.3 Form Tambah Buku

Form menggunakan checkbox kategori berjenjang (induk dan sub-kategori) serta validasi ISBN melalui Value Object.

Dashboard Perpustakaan Admin Administrator

Tambah Buku

Judul Buku *

Slug

Kosongkan untuk otomatis dari judul

ISBN *

Contoh: 9780306406157

ISBN-10 atau ISBN-13, checksum divalidasi Value Object.

Penulis

- Pilih Penulis -

Penerbit

- Pilih Penerbit -

Tahun Terbit

Bahasa

Indonesia

Stok *

0

Batas Stok Menipis

2

Buku aktif (boleh dipinjam)

Deskripsi

Kategori (Many-to-Many, pivot added_at)

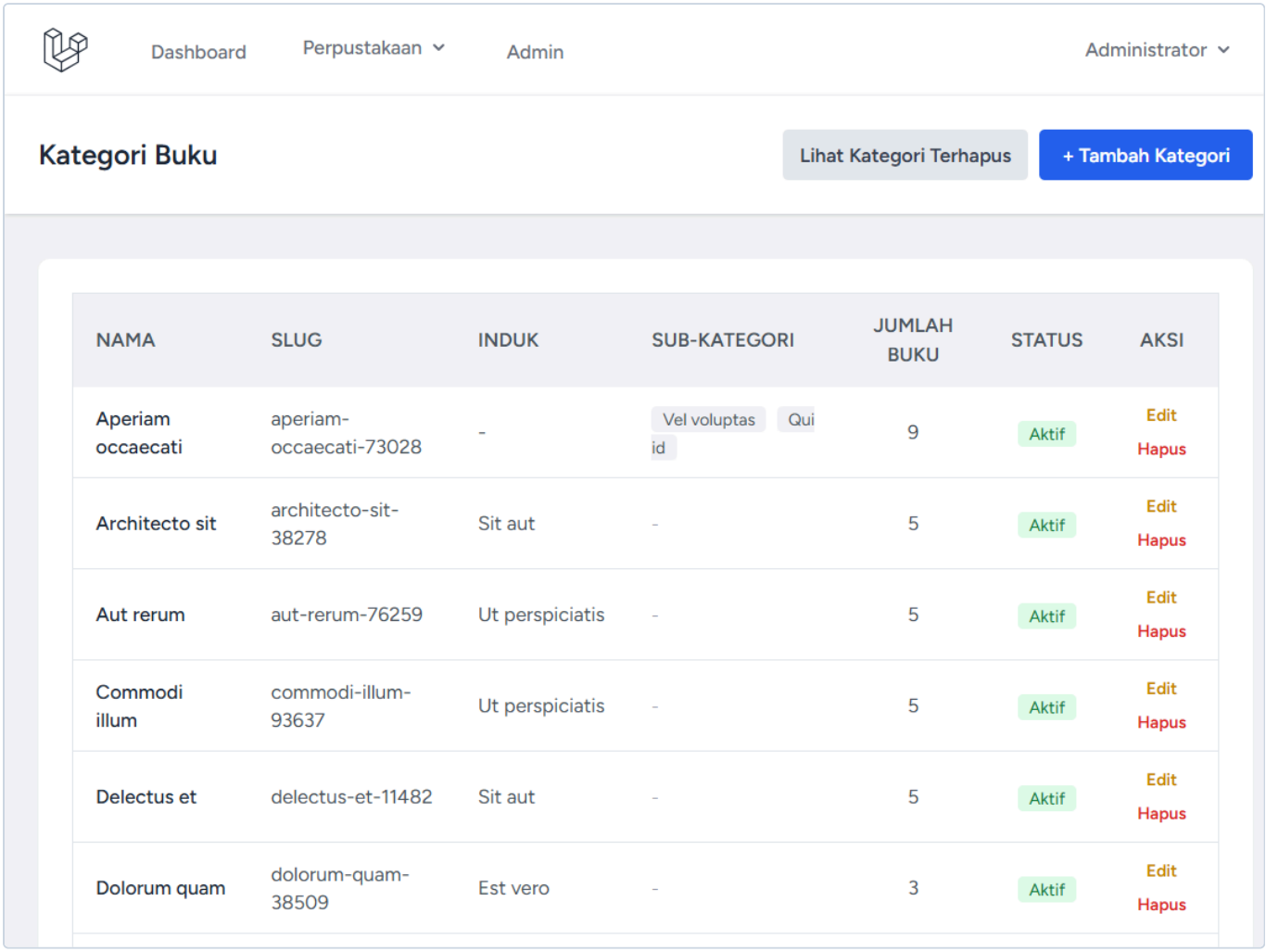
- Aperiam occaecati
- — Qui id
- Aut rerum
- Delectus et
- Est vero
- — Dolorum quam
- Id animi
- — Et placeat
- Provident esse
- Sit aut
- — Delectus et
- — Commodi illum
- Vel voluptas
- — Vel voluptas
- Architecto sit
- Commodi illum
- Dolorum quam
- — Id animi
- Et placeat
- Ipsam odio
- — Provident esse
- Qui id
- — Architecto sit
- Ut perspiciatis
- — Aut rerum

Simpan Buku Batal

Gambar 4.3 — Form tambah buku dengan kategori Many-to-Many.

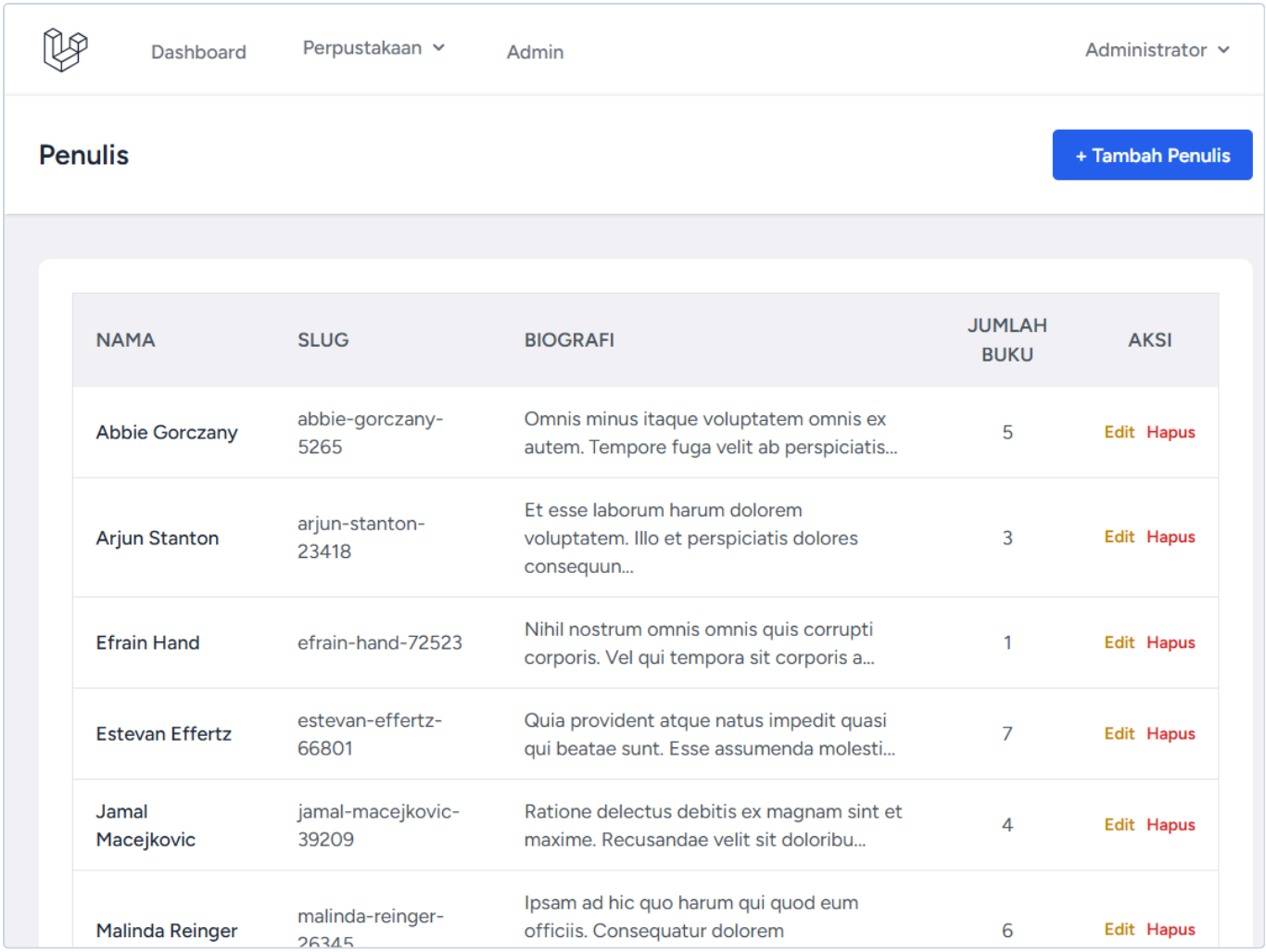
4.3.4 Kategori (Self-Referencing)

Kategori memiliki induk dan sub-kategori. Halaman ini juga menampilkan jumlah buku per kategori.

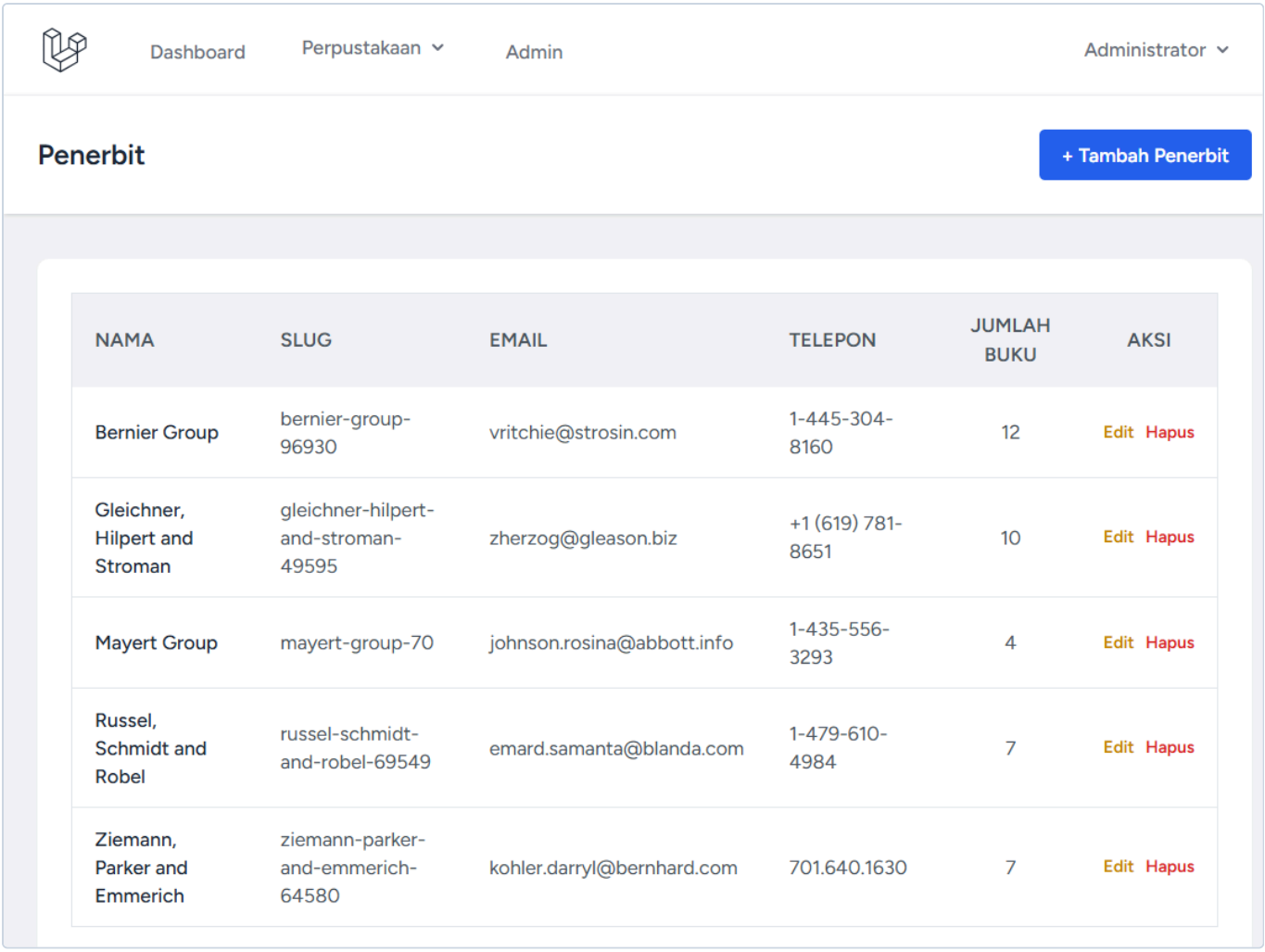


Gambar 4.4 — Kategori dengan relasi self-referencing dan jumlah buku.

4.3.5 Penulis dan Penerbit



Gambar 4.5 — Daftar penulis beserta jumlah buku.

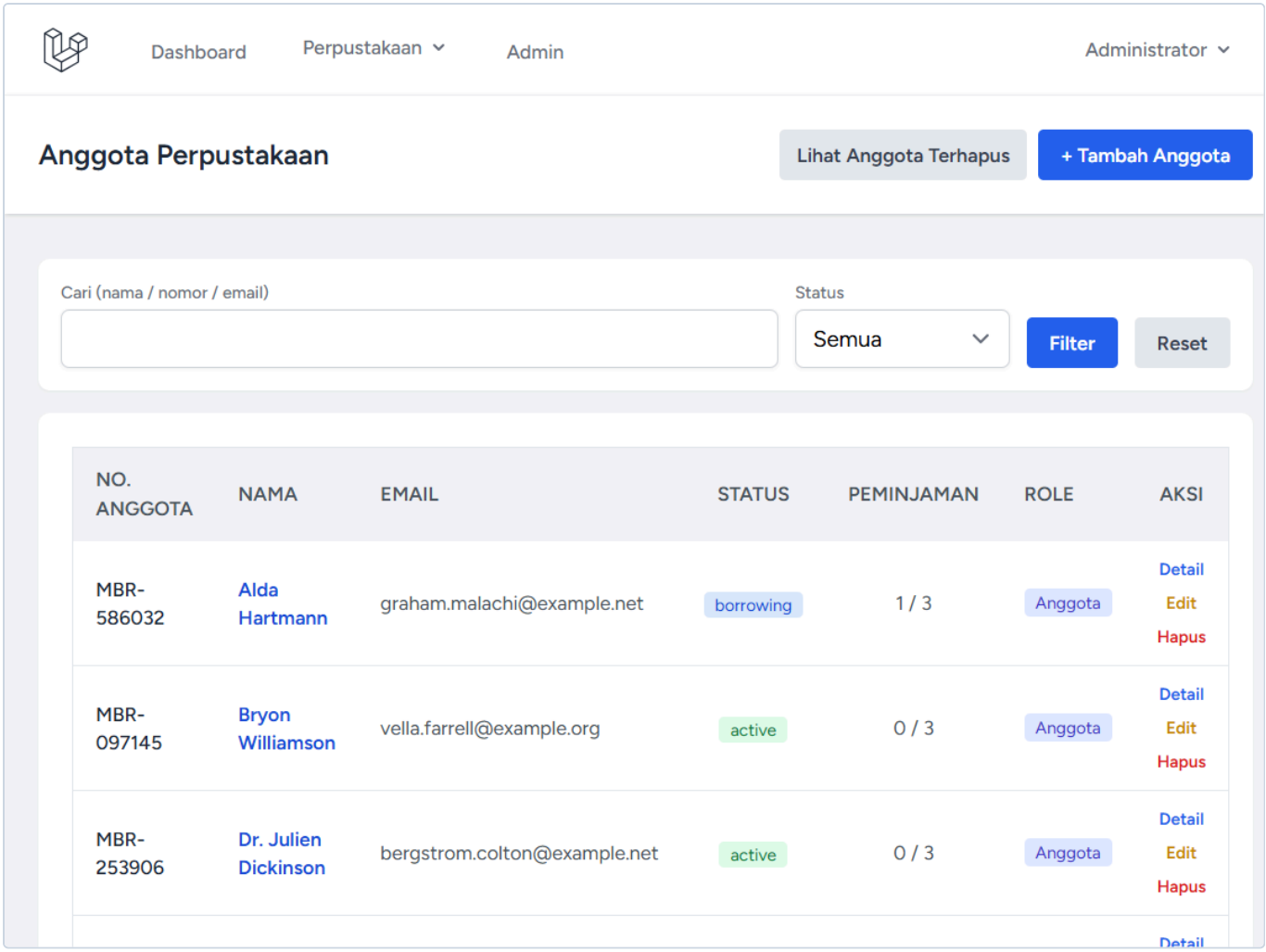


Gambar 4.6 — Daftar penerbit beserta jumlah buku.

4.4 Implementasi Membership

4.4.1 Daftar Anggota

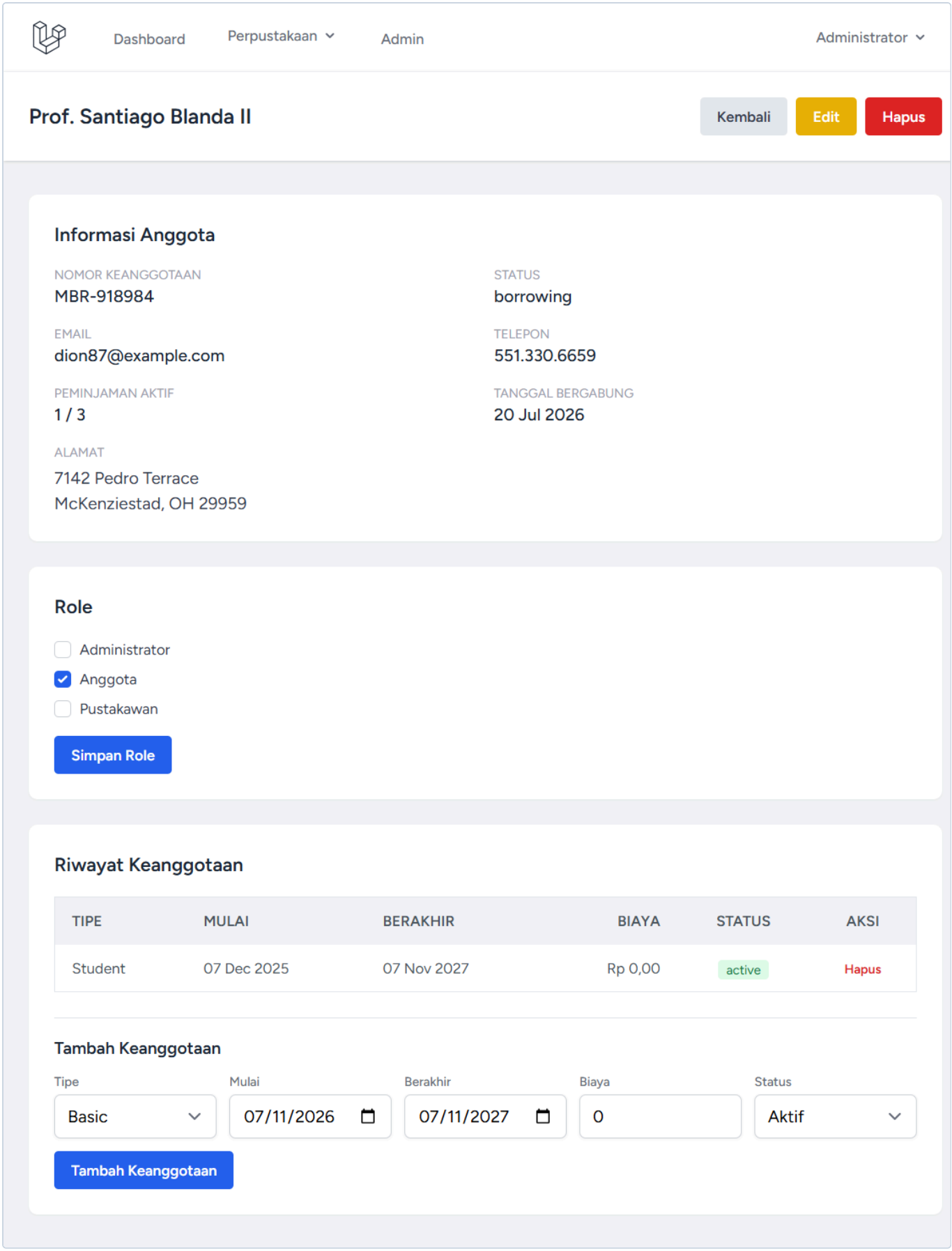
Menampilkan nomor keanggotaan, status, kuota peminjaman, dan role. Relasi `memberships` dan `roles` di-eager load.



Gambar 4.7 — Daftar anggota dengan status dan kuota peminjaman.

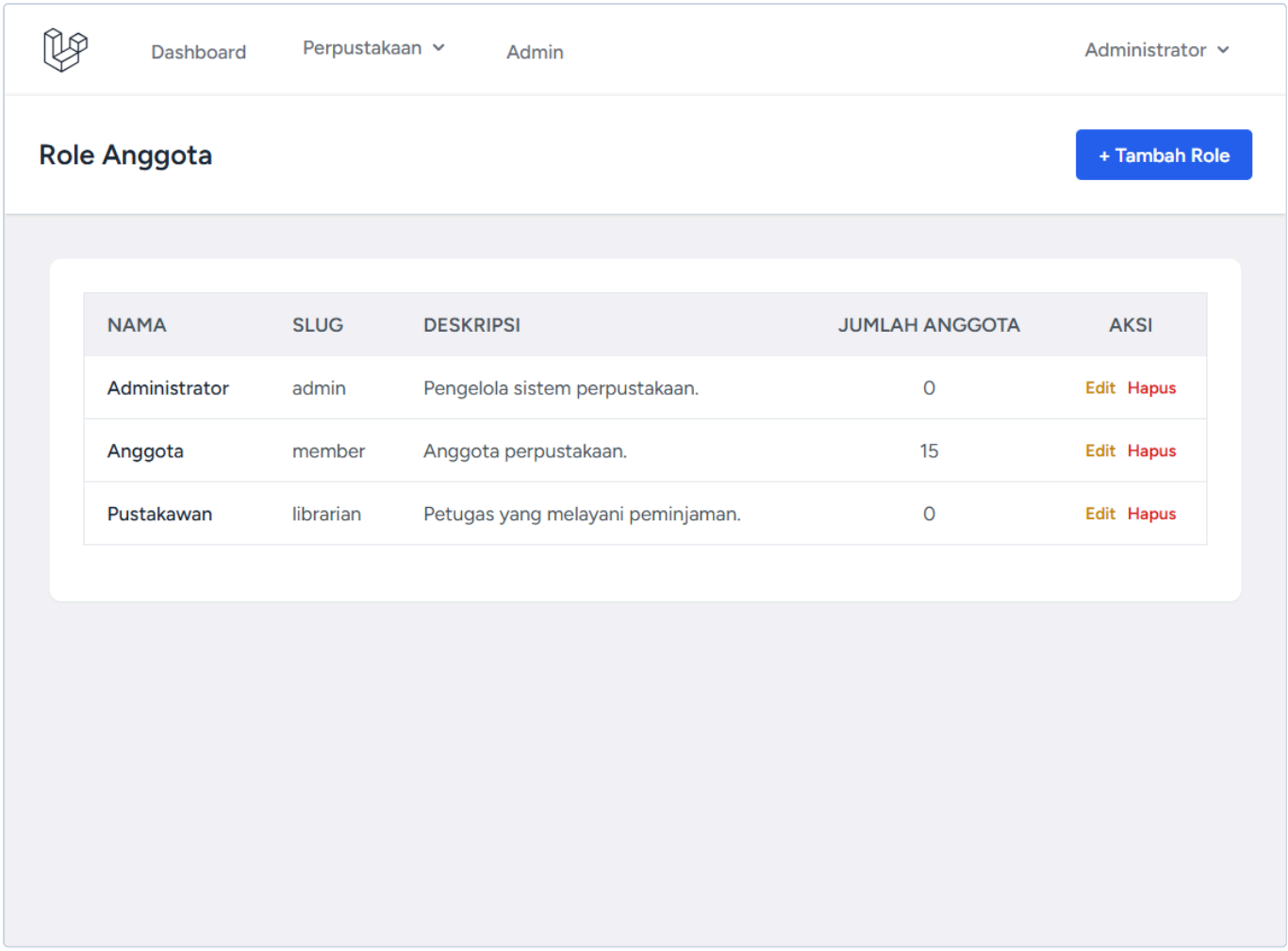
4.4.2 Detail Anggota

Detail anggota menampilkan role (Many-to-Many dengan `assigned_at`) dan riwayat keanggotaan (HasMany), serta form penambahan keanggotaan.



Gambar 4.8 — Detail anggota, role, dan riwayat keanggotaan.

4.4.3 Role

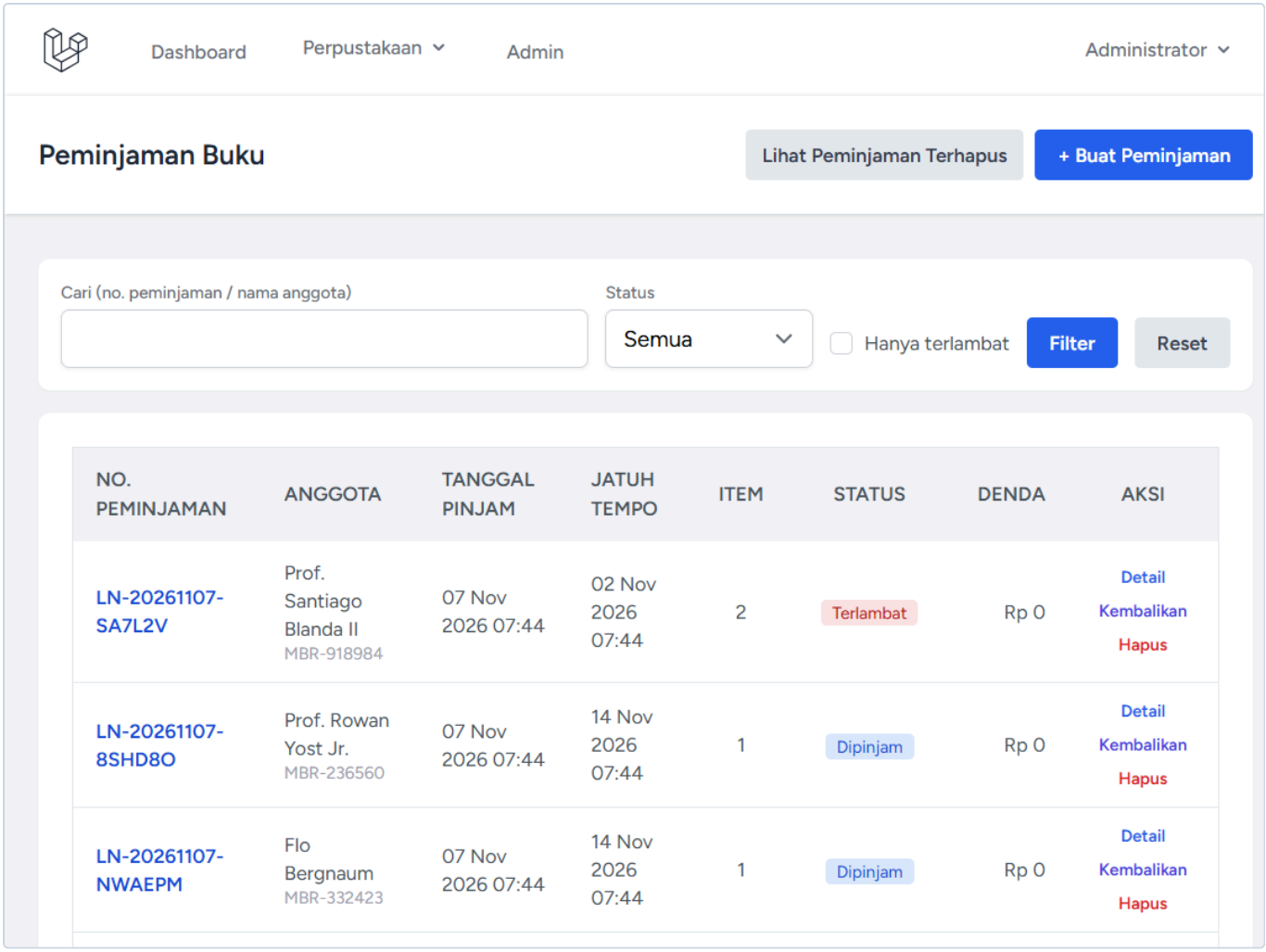


Gambar 4.9 — Daftar role beserta jumlah anggota.

4.5 Implementasi Circulation

4.5.1 Daftar Peminjaman

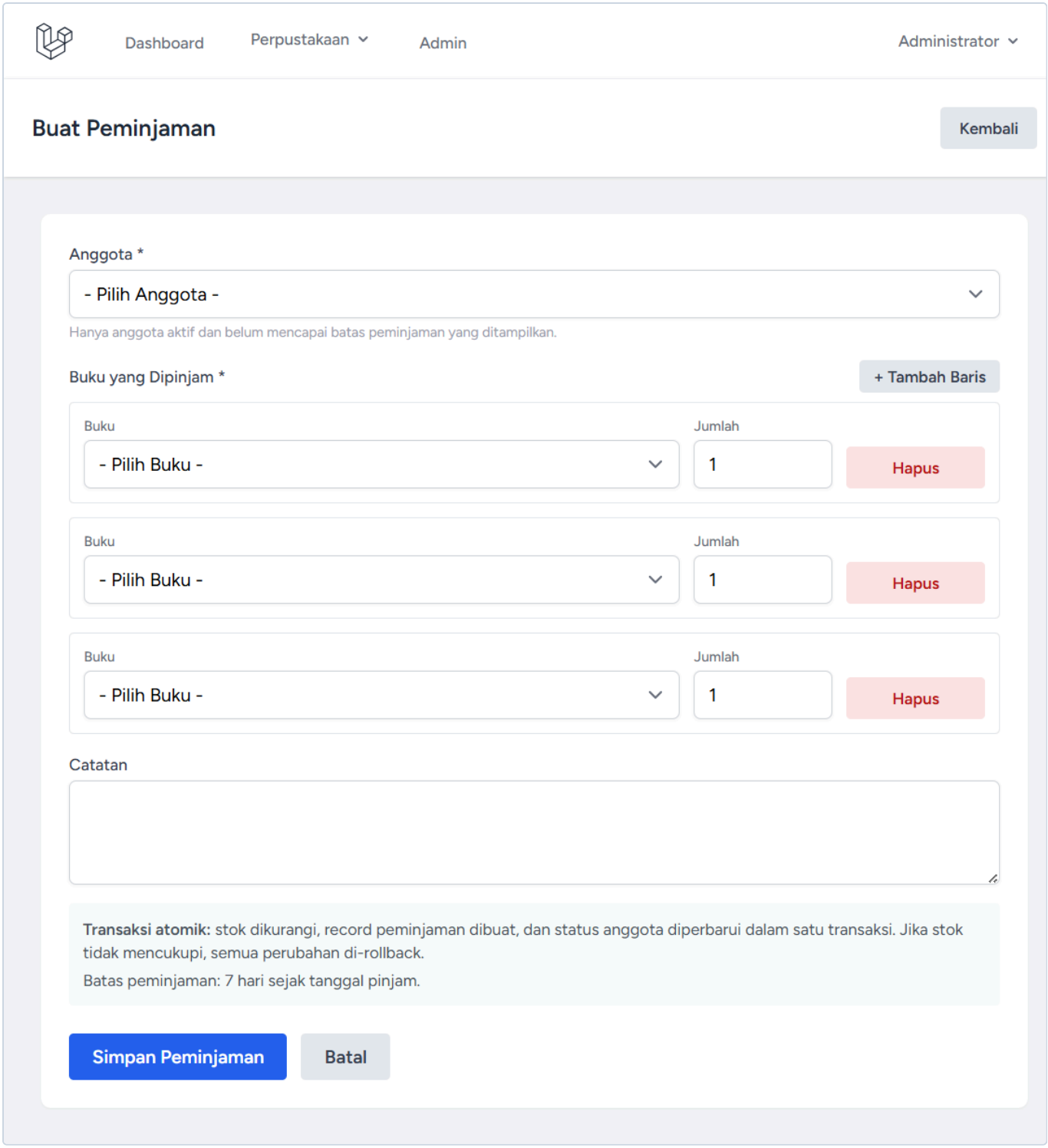
Menampilkan nomor peminjaman, anggota (snapshot), tanggal pinjam, jatuh tempo, jumlah item, status, dan denda. Peminjaman yang melewati jatuh tempo ditandai **Terlambat**.



Gambar 4.10 — Daftar peminjaman; baris pertama berstatus Terlambat.

4.5.2 Form Peminjaman

Form hanya menampilkan anggota yang eligible dan buku yang tersedia. Pengguna dapat menambah beberapa baris buku.



Gambar 4.11 — Form peminjaman dengan beberapa baris buku.

4.5.3 Detail Peminjaman

Detail menampilkan snapshot anggota, status, jatuh tempo, total denda, item (snapshot judul/ISBN), daftar denda, dan riwayat status.

Peminjaman LN-20261107-SA7L2V

Dibuat 07 Nov 2026 07:44

Kembali | Kembalikan Buku

Informasi Peminjaman

ANGGOTA (SNAPSHOT)
Prof. Santiago Blanda II
MBR-918984 · dion87@example.com

STATUS
Terlambat

TANGGAL PINJAM
07 Nov 2026 07:44

JATUH TEMPO
02 Nov 2026 07:44

TANGGAL KEMBALI
-

TOTAL DENDA
Rp 0

CATATAN
Seeder: peminjaman aktif

Ringkasan

Jumlah judul: **2**

Jumlah eksemplar: **2**

Denda belum dibayar: **Rp 0**

Catatan pengembalian

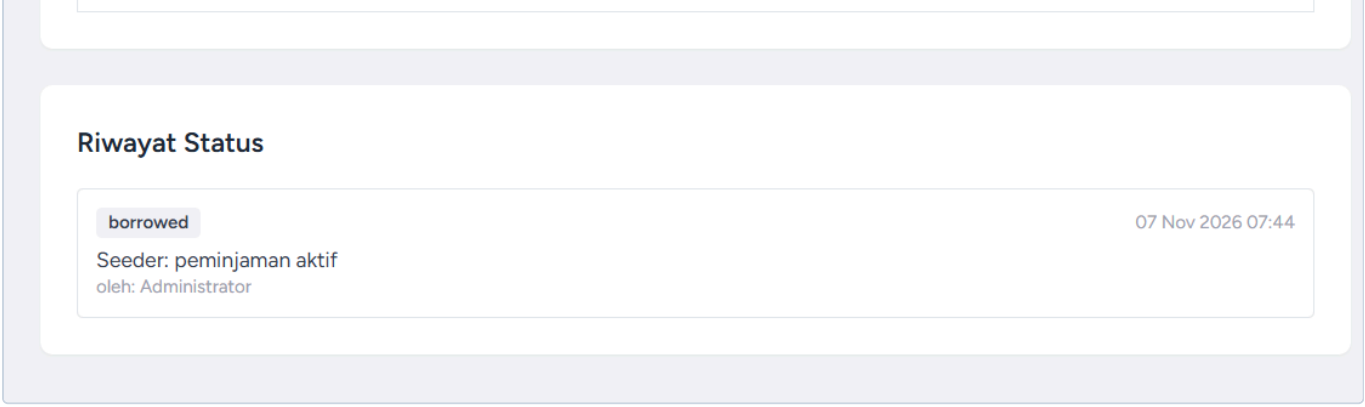
Proses Pengembalian

Detail Peminjaman (snapshot)

JUDUL BUKU	ISBN	JUMLAH
Pariatur voluptas aut sequi.	9789402967517	1
Tempora in.	9783186089342	1

Denda

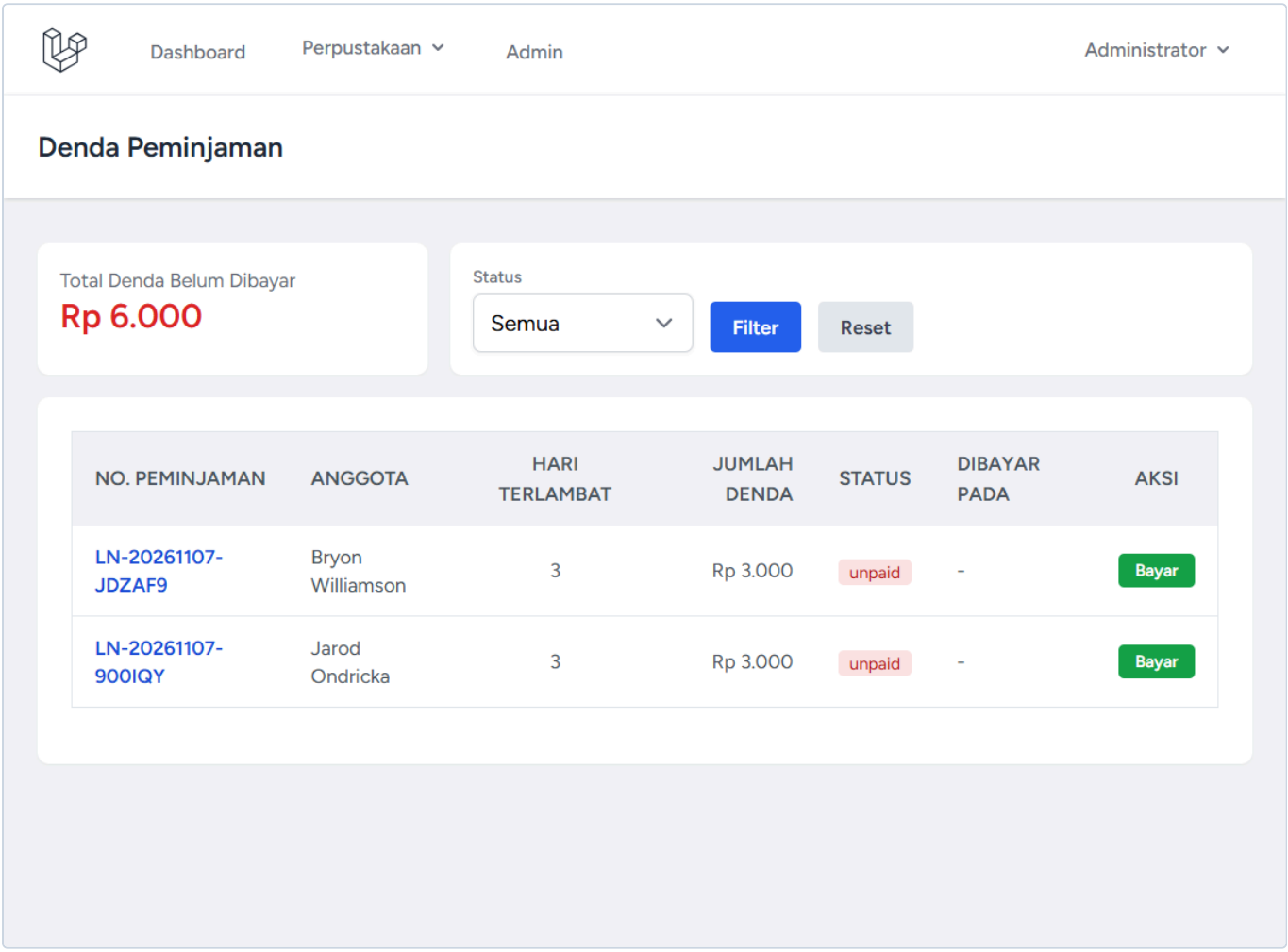
HARI	JUMLAH	STATUS	AKSI
Tidak ada denda.			



Gambar 4.12 — Detail peminjaman: status Terlambat, dua item, dan riwayat status.

4.5.4 Denda

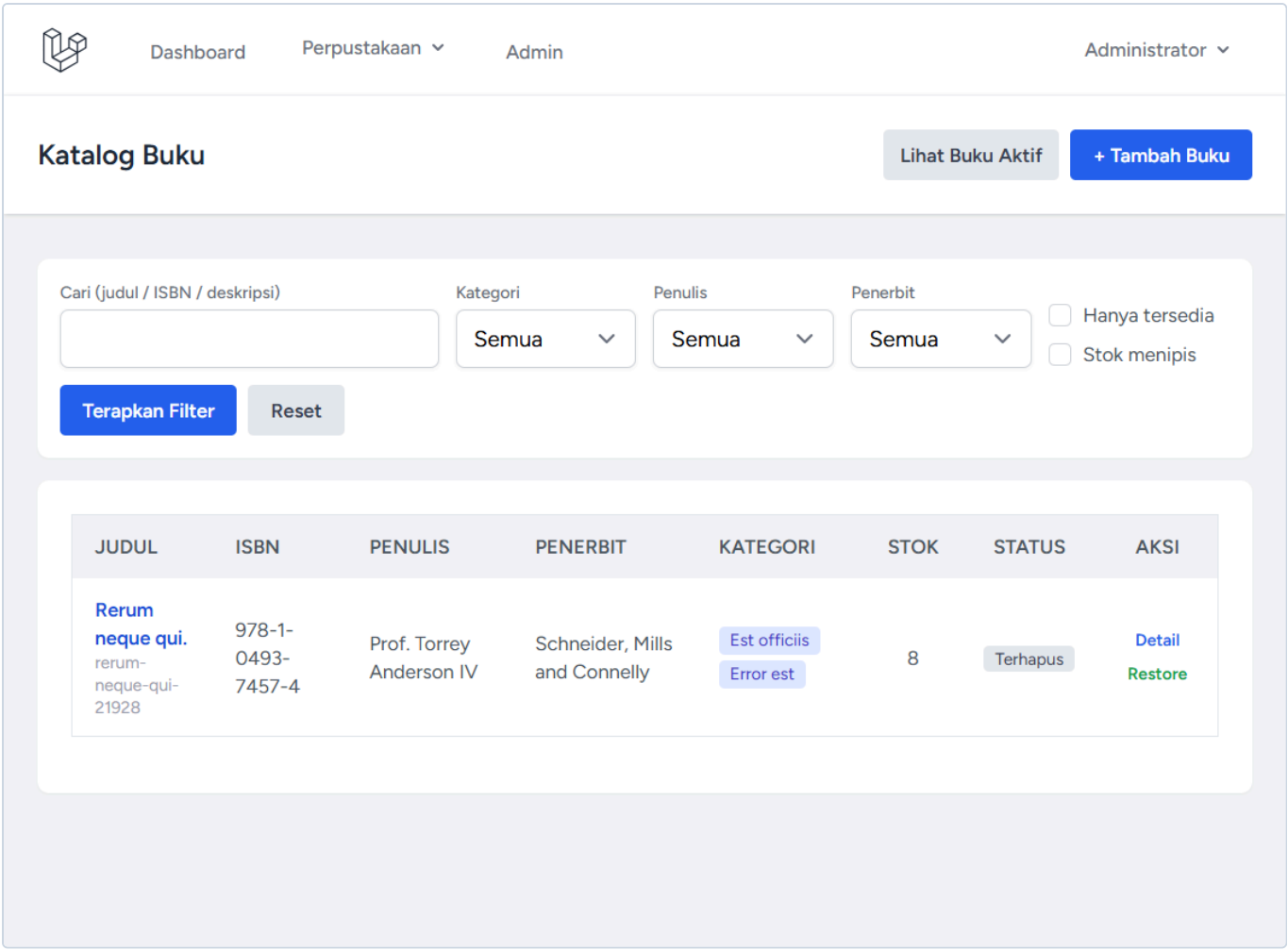
Dua peminjaman yang dikembalikan terlambat 3 hari menghasilkan denda Rp 3.000 masing-masing (total Rp 6.000). Admin dapat menandai denda sebagai lunas.



Gambar 4.13 — Daftar denda dengan total belum dibayar Rp 6.000.

4.6 Soft Delete dan Restore

Buku, anggota, peminjaman, dan kategori menggunakan `SoftDeletes`. Admin dapat melihat data terhapus dan memulihkannya.



Gambar 4.14 — Daftar buku terhapus dengan tombol Restore.

Implementasi pada model:

```
use Illuminate\Database\Eloquent\SoftDeletes;

class Book extends Model
{
    use SoftDeletes;

    // ...
}
```

Route restore:

```
Route::post('books/{id}/restore', [BookController::class, 'restore'])->name('books.restore');
```

4.7 Value Object ISBN

`ISBN` menormalisasi input (menghapus spasi/tanda hubung), memvalidasi checksum ISBN-10/ISBN-13, dan memastikan ISBN-13 berawalan 978/979.

```
final class ISBN
{
    public function __construct(string $value)
    {
        $normalized = strtoupper(preg_replace('/[\s\-]/', '', trim($value)) ?? '');

        if (! preg_match('/^(?:\d{9}[\dX]|\d{13})$/', $normalized)) {
            throw new InvalidArgumentException("Format ISBN tidak valid: '{$value}'.");
        }

        // validasi checksum ISBN-10 / ISBN-13 ...
    }
}
```

Contoh valid: `978-0-306-40615-7` → `9780306406157`; `0-8044-2957-X` → `080442957X`.

4.8 Value Object Denda

Denda dihitung otomatis dengan tarif Rp 1.000 per hari dan pembulatan ke atas untuk keterlambatan sebagian hari.

```
public static function between(
    DateTimeInterface $dueAt,
    DateTimeInterface $returnedAt,
    float $ratePerDay = self::DEFAULT_RATE_PER_DAY,
): self {
    $secondsLate = $returnedAt->getTimestamp() - $dueAt->getTimestamp();

    if ($secondsLate <= 0) {
        return self::fromDaysLate(0, $ratePerDay);
    }

    $daysLate = (int) ceil($secondsLate / 86400);

    return self::fromDaysLate($daysLate, $ratePerDay);
}
```

Contoh: terlambat 3 hari → `3 × Rp 1.000 = Rp 3.000`.

4.9 Transaksi Atomik dan Rollback

`LoanService@borrow` membungkus seluruh operasi dalam `DB::transaction`. Stok dikurangi dengan `lockForUpdate()` untuk mencegah race condition.

```
return DB::transaction(function () use ($memberId, $items, $notes, $changedBy) {
    $member = $this->memberService->getMemberForLoan($memberId);

    if (! $this->memberService->isEligibleToBorrow($memberId)) {
        throw new MemberNotEligibleException(/* ... */);
    }

    $loan = Loan::create([/* ... */]);

    foreach ($items as $item) {
        $book = $this->bookService->getBookForLoan($bookId);

        if (! $this->bookService->isStockAvailable($bookId, $quantity)) {
            throw new InsufficientStockException($book['title'], $book['stock'], $quantity);
        }

        $this->bookService->reduceStock($bookId, $quantity);
        LoanItem::create([/* snapshot */]);
    }

    $this->memberService->markAsBorrowing($memberId);
    LoanStatusHistory::create([/* ... */]);
    event(new BookBorrowed($loan));
});
```

Feature test memverifikasi bahwa ketika stok tidak cukup: `loans`, `loan_items`, stok, dan status anggota tidak berubah.

4.10 Domain Event dan Listener

Event `BookBorrowed` dan `BookReturned` ditangani listener yang menulis ke `storage/logs/library-events.log`.

```
class BookBorrowed
{
    public function __construct(public Loan $loan) {}
}

class LogBookBorrowed
{
    public function handle(BookBorrowed $event): void
    {
        Log::channel('library')->info('BookBorrowed', [
            'loan' => $event->loan->loan_number,
            'member' => $event->loan->member_name,
            'items' => $event->loan->items->count(),
            'borrowed_at' => $event->loan->borrowed_at->toDateTimeString(),
            'due_at' => $event->loan->due_at->toDateTimeString(),
        ]);
    }
}
```

Contoh isi log:

```
[2026-11-07 07:44:30] BookReturned | loan=LN-20261107-JDZAF9 | member=Bryon Williamson (#7)
    | returned_at=2026-11-07 07:44:30 | days_late=3 | fine=Rp 3.000
[2026-11-07 07:44:30] BookBorrowed | loan=LN-20261107-7IUKFF | member=Dr. Wilber Kulas II (#11)
    | items=1 | borrowed_at=2026-11-07 07:44:30 | due_at=2026-11-14 07:44:30
```

4.11 Optimasi Query

Perintah `php artisan library:query-demo` mengukur jumlah query pada data demo:

Skenario	Tanpa eager loading	Dengan eager loading
Daftar peminjaman + item	12	2
Daftar buku + penulis + kategori	81	3

Penurunan: **83%** untuk peminjaman dan **96%** untuk buku. Detail dan output lengkap ada di `docs/QUERY_OPTIMIZATION.md`.

Eager loading diterapkan di seluruh controller daftar/detail, misalnya:

```
$loans = Loan::with(['items', 'fines', 'member'])
    ->withCount('items')
    ->latest('borrowed_at')
    ->paginate(15);

$books = Book::with(['author', 'publisher', 'categories'])
    ->available()
    ->search($keyword)
    ->inCategory($categoryId)
    ->paginate(12);
```

4.12 Pengujian Otomatis

```
Tests:    57 passed (181 assertions)
Duration: ~8.5s
```

Rincian:

File	Jumlah fokus	Hasil
`tests/Unit/IsbnTest.php`	7 test ISBN	Lulus
`tests/Unit/DendaTest.php`	6 test denda	Lulus
`tests/Feature/Library/LoanTransactionTest.php`	5 test transaksi/event	Lulus
`tests/Feature/Library/SoftDeleteRestoreTest.php`	7 test soft delete/otorisasi	Lulus
`tests/Feature/Library/QueryOptimizationTest.php`	4 test query/scope	Lulus
`tests/Feature/Library/PageSmokeTest.php`	2 test smoke halaman	Lulus
Test bawaan Breeze/Products	—	Lulus

Total **57 test, 181 assertion, semuanya lulus**.

4.13 Pengujian Manual

No	Skenario	Hasil
1	Login admin dan akses seluruh menu	Berhasil
2	Login user biasa; akses `/books/create`	403 Forbidden
3	Filter/pencarian buku	Berhasil, query string dipertahankan
4	Buat peminjaman (anggota eligible + buku tersedia)	Berhasil; stok berkurang, status anggota berubah
5	Rollback stok tidak cukup	Berhasil; tidak ada data tersisa
6	Pengembalian tepat waktu	Berhasil; stok kembali, denda 0
7	Pengembalian terlambat 3 hari	Berhasil; denda Rp 3.000
8	Bayar denda	Berhasil; status menjadi `paid`
9	Soft delete + restore buku	Berhasil
10	Lihat halaman terhapus anggota/peminjaman	Berhasil
11	Log event `BookBorrowed` / `BookReturned`	Bertambah sesuai aksi
12	Demo query N+1 vs eager	12 vs 2; 81 vs 3

4.14 Kendala dan Solusi

Kendala	Penyebab	Solusi
Factory model `Modules\*` tidak ditemukan	Resolver default mengasumsikan namespace `App\Models`	`Factory::guessFactoryNamesUsing()` di `AppServiceProvider`
`/books/create` 404	Route `books/{book}` terdaftar lebih dulu	Group route admin (create/store) didaftarkan sebelum route read-only
Denda hasil seeder selalu 0	`Collection::slice()` mempertahankan key sehingga indeks bukan 0	Tambahkan `->values()` sebelum loop
Halaman kategori 500	Type hint `Illuminate\Http\Request` tanpa import di namespace modul	Tambahkan `use Illuminate\Http\Request;`
Menu dropdown tidak sejajar	Trigger dropdown tidak memiliki tinggi penuh nav	Tambahkan `h-16 items-center` pada tombol trigger
Build Vite gagal	Optional native binding `@rolldown/binding-win32-x64-msvc` belum ada	`npm install` lalu `npm run build`
`npm.ps1` diblokir	Execution policy PowerShell	Gunakan `npm.cmd`

BAB V — PENUTUP

5.1 Kesimpulan

1. Sistem perpustakaan berhasil dibangun sebagai **modular monolith** dengan tiga bounded context (Catalog, Membership, Circulation) dan Shared Kernel berisi contract.
2. Relasi lanjutan berhasil diterapkan: Many-to-Many `book_category` dengan `added_at`, self-referencing kategori, HasMany `Loan → LoanItem`, serta Many-to-Many `member_role`.
3. Transaksi database menjamin konsistensi peminjaman: stok, record peminjaman, detail item, dan status anggota berubah bersama; kegagalan stok menyebabkan rollback total.
4. Eager loading menurunkan jumlah query dari 12 menjadi 2 (peminjaman) dan 81 menjadi 3 (buku) pada data demo.
5. Soft delete/restore, domain event, dan Value Object ISBN/Denda berfungsi dan terverifikasi melalui 57 test otomatis (181 assertion).

5.2 Saran

1. Tambahkan antrean (queue) untuk notifikasi domain event agar tidak membebani request.
2. Pisahkan database per context bila skala aplikasi tumbuh (langkah menuju microservices).
3. Tambahkan kebijakan (policy) yang lebih granular daripada middleware role.
4. Integrasikan payment gateway untuk pembayaran denda.
5. Tambahkan observability (metrics/log terstruktur) untuk memantau performa query di produksi.

DAFTAR PUSTAKA

1. Evans, E. (2003). *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Addison-Wesley.
2. Fowler, M. (2014). *Microservices: A Definition of This New Architectural Term*.
3. Laravel Documentation. *Eloquent: Relationships*. https://laravel.com/docs/eloquent-relationships
4. Laravel Documentation. *Database: Query Builder & Transactions*. https://laravel.com/docs/queries
5. Laravel Documentation. *Events*. https://laravel.com/docs/events
6. Laravel Documentation. *Soft Deletes*. https://laravel.com/docs/eloquent#soft-deleting
7. Module Praktikum Backend Development Pertemuan 6 (MP-BED-06).

LAMPIRAN

A. Akun Demo

Peran	Email	Password
Admin	admin@example.com	password123
User	user@example.com	password123

B. Perintah Penting

```
php artisan migrate:fresh --seed
php artisan test
php artisan library:query-demo
php artisan route:list
php artisan serve
```

C. Daftar Screenshot

File	Keterangan
`01-katalog-buku.png`	Katalog buku + filter
`02-detail-buku.png`	Detail buku + pivot `added_at`
`03-form-tambah-buku.png`	Form tambah buku
`04-kategori.png`	Kategori self-referencing
`05-penulis.png`	Daftar penulis
`06-penerbit.png`	Daftar penerbit
`07-anggota.png`	Daftar anggota
`08-detail-anggota.png`	Detail anggota + role
`09-role.png`	Daftar role
`10-peminjaman.png`	Daftar peminjaman
`11-form-peminjaman.png`	Form peminjaman
`12-detail-peminjaman.png`	Detail peminjaman + item
`13-denda.png`	Daftar denda
`14-buku-terhapus.png`	Soft delete + restore

D. Struktur Bukti Repositori

```
auth-monolith/
├── app/Modules/            # implementasi bounded context
├── tests/                  # unit & feature test
├── docs/
│   ├── ARSITEKTUR.md
│   ├── QUERY_OPTIMIZATION.md
│   ├── PANDUAN_PENGUJIAN.md
│   └── screenshots/
├── LAPORAN.md
├── LAPORAN.pdf
└── README.md
```